Week 7

Modeling Languages II: The Mathematical Programming View

◎ Learning Objectives

By the end of this chapter, you will be able to:

- **(H1)** define a linear program (LP) and an integer/ mixed-integer program (IP/MIP) formally in terms of decision variables, a linear objective function and linear constraints; explain what an LP relaxation of a MIP is and why solving it produces a valid bound on the MIP's optimal objective value;
- **(H2)** explain, at a conceptual level, how the branch-and-bound algorithm uses an LP relaxation's bound at each search-tree node to decide when a branch can be pruned without exploring it further, and why this guarantees a provably optimal integer solution when the search terminates;
- **(H3)** build and run first working models in `gurobipy`: declare continuous, integer and binary decision variables, add linear constraints and a linear objective, call `optimize()`, and read the solution status and values – for the assignment, knapsack and production-planning problems;
- **(H4)** compare a CP-SAT formulation and a `gurobipy` formulation of the same problem, identify what each modeling paradigm expresses more directly, and state a working rule of thumb for choosing between a constraint-programming and a mathematical-programming solver for a new problem;
- **(H5)** translate a paper LP/MIP formulation (H1) into a verified, running `gurobipy` program (H3) and interpret the solver's reported status and objective value against the formal LP/MIP definition (H1) – closing the loop from theory to a working solver run, the same pattern Week 1's H5 established for CP-SAT.

Weeks 1 through 6 built every model in this course as a constraint satisfaction problem, handed to OR-Tools CP-SAT directly or to MiniZinc's declarative language, run unchanged on Gecode and, briefly, on a MIP backend. This week opens that MIP backend directly: you write linear and integer programs by hand, understand LP relaxation and branch-and-bound as the mechanics that make a MIP solver provably optimal, and build your first `gurobipy` models – a second full paradigm, not a passing mention, standing alongside CP-SAT for the rest of the course.

Four units carry this chapter. Unit 1 defines a linear program, an integer/mixed-integer program, and LP relaxation, and works Unit 1's own knapsack instance by hand. Unit 2 builds directly on that relaxation to explain branch-and-bound – how a MIP solver proves an integer solution optimal by bounding and pruning, traced by hand on the exact same knapsack instance. Unit 3 turns both units' paper mathematics into real, running `gurobipy` code across three problems: knapsack, assignment, and a brand-new production-planning problem. Unit 4 closes the chapter by placing a CP-SAT sketch and Unit 3's `gurobipy` model of the same assignment problem side by side, building a working rule of thumb for choosing between the two paradigms.

This chapter deliberately stays out of several topics later weeks own outright. It names, but does not define, compute or interpret, the dual values and reduced costs (`Pi` and `RC`) `gurobipy` can report on a solved model – that full treatment is Week 10's subject. It adds no objective function to CP-SAT – Week 8 is where CP-SAT gets `Minimize`/`Maximize` for the first time. It diagnoses no infeasibility (Week 15), and it introduces no scheduling-specific constructs such as interval variables or `Cumulative` (Week 11). Knapsack and assignment are not new problems here – Week 6 already modeled and solved both in MiniZinc on three solvers – this week reuses the exact same two problems, modeled a third way, directly in `gurobipy` instead of through MiniZinc's declarative layer, so you recognize them immediately.

7.1 Linear and Integer Programs, and LP Relaxation

1 • Pre-instruction **Why It Matters**

Every model you have built so far this course answers a single question: is there a solution, or here are all of them. Week 8 will eventually add a `Minimize`/`Maximize` to CP-SAT, but the field of mathematical programming has been objective-first from the start. Imagine a factory manager who does not want *a* production plan that respects the labour and material limits – the manager wants provably the *most profitable* plan, and proof that no better plan exists. This unit gives you the language that question is written in.

1 • Pre-instruction **Prior Knowledge**

Week 1 Unit 1's formal CSP triple – variables, domains, constraints – is the reference point this unit contrasts against. An LP keeps "variables" and "constraints" but replaces finite domains with continuous ranges and adds a linear objective as a required fourth ingredient. Week 6's MiniZinc chapter already ran one model on a MIP backend (Gurobi or HiGHS) without ever showing what that backend was doing internally; this unit is the first time that internal machinery – LP relaxation – is named and defined.

Recall a constraint satisfaction problem's variables, domains and constraints from Week 1 Unit 1. Nothing in that triple asks for "better" – a CSP either has a solution or it does not, and every solution counts equally. Mathematical programming starts from a different place entirely: it asks for the best assignment, according to a stated objective, among every assignment that respects the constraints. This unit builds that idea up from the ground, one ingredient at a time.

7.1.1 Linear programs: continuous variables, a linear objective

linear program (LP): An optimization problem with continuous decision variables, a linear objective function, and linear (in)equality constraints.

A **linear program (LP)** has three ingredients. First, its decision variables are continuous by default – each one may take any real value in some range, not only the handful of discrete values a CSP's domain allowed. Second, it carries a linear objective function – a weighted sum of the decision variables that the model is asked to minimize or maximize, and this objective is not optional the way a constraint optimization problem's objective was an add-on to a CSP (Week 1 Unit 1 mentioned COPs only in passing); an LP is objective-first by definition. Third, it carries linear constraints – inequalities ($\leq$ or $\geq$) or equalities ($=$) on weighted sums of the variables – together with non-negativity, the usual default requirement that every decision variable stay at or above zero unless a model says otherwise.

Definition: Linear program (LP)

An optimization problem with continuous decision variables, a linear objective function, and linear (in)equality constraints.

The **objective function** is this weighted sum itself – the exact quantity `setObjective` takes in Unit 3's code, and the number a solver reports back once it has found the best value that quantity can reach under the constraints.

Analogy

Think about a travel budget spreadsheet where every line item is fully divisible – you could book 2.5 nights in a hotel, or allocate exactly 37.20 currency units to meals, and the spreadsheet does not object. That freedom to take any fractional value, as long as the totals stay inside your budget rules, is exactly what a linear program's continuous variables give you.

7.1.2 Integer and mixed-integer programs

integer program (IP): A linear program in which every decision variable is restricted to integer values.

Real decisions are frequently not divisible this way – you cannot ship half a manufactured unit, and a worker either is or is not assigned to a task. An **integer program (IP)** is a linear program in which every decision variable is restricted to integer values; a **mixed-integer program (MIP)** mixes continuous and integer (or binary) variables in one model – some quantities stay fully divisible while others must land on a whole number. A particularly common special case restricts a variable to exactly two values, `{0,1}`, called a **binary variable** because it represents a yes/no or include/exclude decision – exactly the kind of decision the knapsack and assignment problems later this week both need: does this item go in the knapsack, does this worker do this task.

Analogy

Return to the travel budget: a "meals" line can be any fractional amount, but a "conference registration" line is paid in full or not at all – there is no such thing as half a registration. Mixing a fully divisible line and an all-or-nothing line in one spreadsheet is exactly what a mixed-integer program mixes in one model: some variables stay continuous, others are pinned to whole numbers or to `{0,1}`.

7.1.3 The feasible region, and where an LP's optimum sits

feasible region: The set of all variable assignments that satisfy every constraint of an LP/MIP.

Every LP or MIP's constraints, taken together, carve out a set of points that satisfy all of them at once – its **feasible region**. For a pure LP (no integrality restrictions), this set has a specific geometric shape: a **polytope** – a flat-sided, possibly unbounded region, cut out by the straight lines (or, in more variables, flat planes) each linear constraint draws. A useful fact about linear programs, stated here without proof, is that an LP's optimum always sits at one of the polytope's corners, its **vertices**, never strictly inside the region – a fact solvers rely on directly, since it means searching only the corners (not the infinitely many interior points) suffices to find the optimum.

Figure 7.1 draws this idea on a small, deliberately separate two-variable instance – a production-style toy with constraints $6x_1 + 4x_2 \leq 24$ and $x_1 + 2x_2 \leq 6$, maximizing $5x_1 + 4x_2$. Note this is *not* the three-item knapsack instance worked below; that instance needs three axes and is reserved for this unit's hand arithmetic instead. The figure's shaded quadrilateral is the feasible region; its LP optimum sits at the vertex $(3, 1.5)$, objective value 21 – fractional in x_2 – while the true integer optimum, restricted to whole-number points inside the same region, sits at the different vertex $(4, 0)$, objective value 20. Hold onto that gap between 21 and 20; it is exactly the idea the next subsection formalizes.

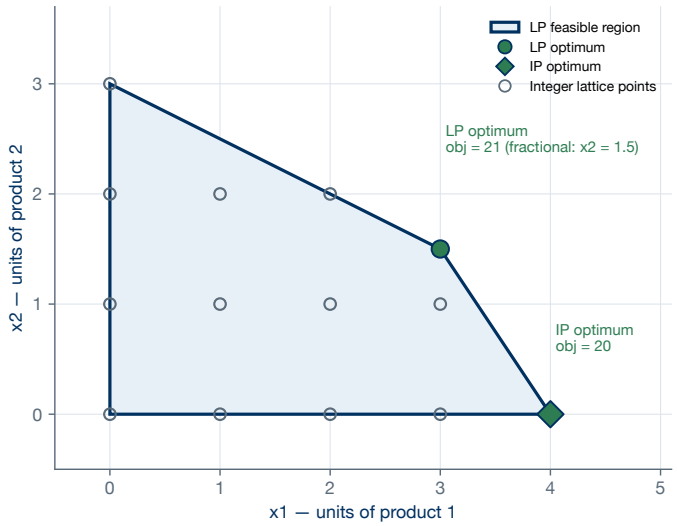


Figure 7.1: LP relaxation feasible region: fractional LP optimum vs. true integer optimum.

7.1.4 LP relaxation: a bound, not an answer

LP relaxation: The LP obtained from a MIP by dropping every integrality restriction, keeping the same objective and constraints; its optimal value is a valid bound on the MIP's optimal value.

Given a MIP, dropping every integrality restriction – turning every `integer` or `binary` variable into a `continuous` one, while keeping the same objective and the same constraints – produces its **LP relaxation**. Two facts make this construction useful rather than merely a curiosity. First, solving the relaxation is always at least as easy as solving the original MIP, because a continuous LP can be solved directly (no search over discrete choices is needed). Second, and more importantly, the relaxation's optimal value is always a valid **bound** on the MIP's optimal value – an upper bound when maximizing, a lower bound when minimizing. The argument is short: every point that is feasible for the original MIP (it satisfies the integrality restrictions *and* every constraint) is automatically feasible for the relaxed LP too (the relaxation only removed restrictions, it added none), so the best the relaxed

LP can achieve is at least as good as the best the MIP can achieve. The relaxation, in short, can never do worse than the MIP – which is exactly why Figure 7.1's LP optimum (21) sits above its integer optimum (20): the relaxation is an optimistic estimate, not a guaranteed achievable one.

Case Study 7.1: The A/B/C knapsack: IP formulation and its LP relaxation, by hand

A 0/1 knapsack has capacity 6 and three items: A (weight 2, value 3), B (weight 3, value 4), C (weight 4, value 5). Written as an IP, with one binary variable per item stating whether it is packed:

$$\text{maximize } 3x_A + 4x_B + 5x_C$$

$$\text{subject to } \quad 2x_A + 3x_B + 4x_C \leq 6, \quad x_A, x_B, x_C \in \{0, 1\}.$$

Enumerating all 8 subsets by hand (every combination of packing or not packing each of the three items, checked against the weight-6 capacity) shows the true integer optimum is $\{A, C\}$: weight $2 + 4 = 6$, value $3 + 5 = 8$.
The LP relaxation keeps the objective and the constraint identical and only replaces the domain restriction: $0 \leq x_A, x_B, x_C \leq 1$. An LP relaxation this small can be solved by hand with the greedy value-per-weight rule – rank items by value divided by weight (A: $3/2 = 1.5$, B: $4/3 \approx 1.333$, C: $5/4 = 1.25$) and fill the capacity in that order, taking as much of each item as fits. Take A fully (weight 2, value 3, capacity 4 left); take B fully (weight 3, value 4, capacity 1 left); C's remaining weight of 4 no longer fits whole, so take the fraction that exactly uses the last unit of capacity, $x_C = 1/4 = 0.25$ (weight 1, value 1.25). The relaxed optimum is $3 + 4 + 1.25 = 8.25$ – strictly above the true integer optimum of 8, illustrating directly that a relaxation's bound is not itself an achievable integer answer.

Caution

A frequent misconception at this point is treating an LP relaxation's optimal value as though it were itself a legitimate answer to the original problem. It is not: 8.25 units of value is not a real knapsack packing – no combination of whole items reaches it, and the fractional $x_C = 0.25$ has no real-world meaning for an item that must be packed whole or left out. The relaxation's value is a *ceiling* the true integer optimum cannot exceed, useful precisely because it is easy to compute and always safe to trust as a bound – never because it is itself achievable.

3 • Participation **Now You Try**

Using the knapsack instance above (capacity 6; items A: weight 2 value 3, B: weight 3 value 4, C: weight 4 value 5), write the IP formulation and its LP relaxation, stating exactly which part of the formulation changes between the two. Then compute the LP relaxation's optimum by hand using the greedy value-per-weight rule, stating which variable ends up fractional.

Feedback. IP: maximize $3x_A + 4x_B + 5x_C$ subject to $2x_A + 3x_B + 4x_C \leq 6$, $x_A, x_B, x_C \in \{0, 1\}$. The LP relaxation keeps the objective and the constraint

identical and only replaces $x_A, x_B, x_C \in \{0, 1\}$ with $0 \le x_A, x_B, x_C \le 1$ – the domain restriction is the only part that changes. Sorting by value/weight ratio (A = 1.5, B $\approx$ 1.333, C = 1.25) and filling greedily: $x_A = 1$ (weight 2, 4 left), $x_B = 1$ (weight 3, 1 left), $x_C = 0.25$ (weight 1, 0 left) – value $3+4+1.25 = 8.25$. The fractional variable is x_C.

Check Yourself 7.1

A second 0/1 knapsack instance has capacity 7 and three items: D (weight 3, value 5), E (weight 4, value 6), F (weight 5, value 8). Write the IP formulation and its LP relaxation, then compute the LP relaxation's optimum by hand using the greedy value-per-weight rule, stating which variable ends up fractional.

5 • Follow-through **Transfer — Where Else?**

The relaxation you just computed by hand is not a one-off trick – it is exactly what Gurobi computes internally at every node of the search tree Unit 2 walks through, it is exactly the number that lets a MIP solver *prove* optimality, and this same knapsack instance becomes real, running code in Unit 3's `knapsack.py`. Week 10 later attaches a full sensitivity report to problems built the same way.

7.2 Branch-and-Bound and Why MIP Works

1 • Pre-instruction **Why It Matters**

You have already watched a CP-SAT search prune branches using propagation (Weeks 2–3). A MIP solver's decision variables are not tiny finite domains it can propagate over the same way – between 0 and 1 there are infinitely many values – so its pruning has to come from somewhere else entirely: a number, not a domain-consistency argument. This unit shows exactly where that number comes from and how far it can take you with almost no search at all.

1 • Pre-instruction **Prior Knowledge**

Unit 1's LP relaxation and the A/B/C knapsack instance – capacity 6; items A (weight 2, value 3), B (weight 3, value 4), C (weight 4, value 5) – carry over unchanged into this unit's worked trace. Week 3's search-tree vocabulary (branching, backtracking) is reused here in a new setting.

Unit 1 left you with a gap: the LP relaxation's optimum (8.25) sits above the true integer optimum (8), and a relaxation alone cannot tell you which whole-number packing actually achieves 8. This unit closes that gap with an algorithm that uses the relaxation's bound, over and over, to search only the part of the space that could possibly matter.

7.2.1 The algorithm: solve, branch, bound, prune

branch-and-bound: The search algorithm MIP solvers use: branch on a fractional variable to create subproblems, use each subproblem's LP-relaxation value as a bound, and prune any subproblem whose

The **branch-and-bound** algorithm proceeds one node at a time. At the current node, solve its LP relaxation. If the relaxation's solution is already integer-feasible – every

variable happens to land on a whole number – it is a candidate solution, and it becomes the **incumbent** if it beats the best integer solution found so far anywhere in the search. If instead some variable is fractional, pick one such variable and **branch** on it – create two child subproblems, one adding the restriction that the fractional variable equal 0 and the other adding the restriction that it equal 1 (for a binary variable; a general integer variable branches on "round down or below" versus "round up or above"), and recurse on each child independently.

Definition: Branch-and-bound

The search algorithm MIP solvers use: branch on a fractional variable to create subproblems, use each subproblem's LP-relaxation value as a bound, and prune any subproblem whose bound cannot beat the best integer solution found so far.

bound (branch-and-bound): A subproblem's LP-relaxation value, used to decide whether that subproblem's subtree can be pruned without being explored further.

The second half of the algorithm's name is where the real saving comes from. A child subproblem's LP relaxation value is a **bound (branch-and-bound)** on the best objective value reachable anywhere inside that child's own subtree – Unit 1's argument applies recursively at every node, not only at the root. The moment a child's bound is no better than the current incumbent, its entire subtree can be **pruned** – discarded without solving it any further – because nothing inside that subtree can beat what the search already holds in hand. This is why branch-and-bound needs to explore only a fraction of all possible integer assignments and still returns a provably optimal answer: every discarded branch carries a mathematical guarantee, not a guess, that it held nothing better.

Analogy

Picture comparing travel plans and abandoning one you already know cannot win: once you have a fully booked itinerary at a known total cost, and a half-planned alternative whose cheapest-possible completion (even under the most generous assumptions) is already more expensive, you stop planning that alternative further – you already know it cannot win, without ever finishing it. That "cheapest-possible completion under generous assumptions" is exactly a bound; the fully booked itinerary you keep as your fallback is exactly an incumbent.

7.2.2 Contrast with CP-SAT's search

Both branch-and-bound and the CP-SAT search you watched in Weeks 2–4 build a search tree by branching – the family resemblance is real. The difference is what each one uses to justify pruning a branch. Branch-and-bound prunes using a continuous LP relaxation's numeric bound, computed fresh at every node; CP-SAT prunes using constraint propagation over discrete domains – arc consistency, global-constraint reasoning – shrinking a variable's finite domain until an inconsistency is caught directly. Two different sources feed the same underlying idea: do not explore a branch that provably cannot help. Neither is strictly better in general; they are different tools built for different variable types, continuous-friendly bounds on one side, discrete-friendly domain reasoning on the other.

Case Study 7.2: The A/B/C knapsack, branched and bounded to a proof of optimality

Continue Unit 1's knapsack instance. The root relaxation (value 8.25, Unit 1) is fractional on x_C ($x_C = 0.25$), so branch on it.
Child $x_C = 0$: the relaxation reduces to maximizing $3x_A + 4x_B$ subject to $2x_A + 3x_B \leq 6, 0 \leq x_A, x_B \leq 1$; greedy (ratio A = 1.5 still above ratio B ≈ 1.333) gives $x_A = 1, x_B = 1$ – already integral, using weight $2 + 3 = 5 \leq 6$, value $3 + 4 = \mathbf{7}$.
Child $x_C = 1$: fixing $x_C = 1$ uses weight 4 and value 5, leaving capacity $6 - 4 = 2$; the relaxation maximizes $3x_A + 4x_B$ subject to $2x_A + 3x_B \leq 2, 0 \leq x_A, x_B \leq 1$; greedy (A's ratio is still higher, and B's full weight of 3 no longer fits) gives $x_A = 1, x_B = 0$ – already integral, value $5 + 3 = \mathbf{8}$.
The second child's integral value, 8, becomes the incumbent. The first child's bound, 7, is strictly below that incumbent, so it is pruned – its subtree is discarded without any further branching. No further branching is needed anywhere in the tree: the proven optimum is 8, matching Unit 1's by-hand enumeration of all 8 subsets exactly. Figure 7.2 draws this three-node tree in full.

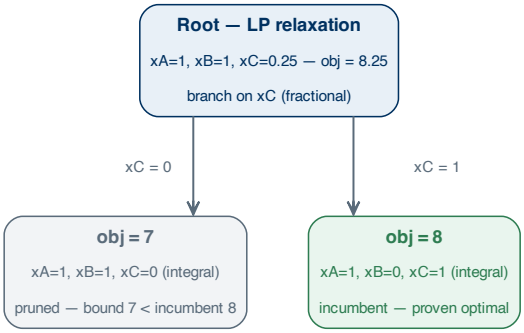


Figure 7.2: Branch-and-bound search tree for the knapsack instance: LP relaxation root, one pruned branch, one proven-optimal incumbent.

Tip

Notice that the $x_C = 0$ child's own relaxation happened to already be integer-feasible – it needed no further branching to become a valid solution. That is a convenience of this particular small instance, not a rule: a real branch-and-bound run frequently must branch several levels deep inside a single subtree before a node's relaxation lands on integer values, and the pruning argument works exactly the same way at any depth, using only that node's own bound against the current incumbent.

7.2.3 What termination guarantees

When branch-and-bound stops – every remaining subtree has either been solved to an integer-feasible node or pruned – the incumbent it holds is not merely the best solution the search happened to try; it is provably optimal. The **optimality gap** between the best remaining bound and the incumbent's value has reached zero, meaning no unexplored part of the search space could possibly contain anything

optimality gap: The (provable) difference between the best bound found and the best incumbent solution's value; a gap of zero means the incumbent is proven optimal.

better – a guarantee, not an estimate.

3 • Participation **Now You Try**

In the branch-and-bound trace above, the $x_C = 0$ child's relaxation turned out to be integer-feasible already, at value 7, and the $x_C = 1$ child's relaxation turned out to be integer-feasible already, at value 8. Explain, in one or two sentences, why it was safe to stop exploring the $x_C = 0$ branch once the $x_C = 1$ branch's value of 8 was known, and why this would still have been safe even if the $x_C = 0$ branch's relaxation had *not* been integral.

Feedback. A branch can be pruned the moment its LP relaxation's value is no better than the current incumbent, because the relaxation's value is an upper bound on everything reachable inside that branch (Unit 1); once the incumbent is 8, a branch whose bound is 7 cannot possibly contain anything better than 8, regardless of whether that bound of 7 happens to already be an integer solution or would need further branching to become one – the pruning argument only depends on the bound, never on whether the bound is itself already integral.

Check Yourself 7.2

In the branch-and-bound example worked in this unit, the $x_C = 0$ child's relaxation was already integer-feasible at value 7. If it had instead been fractional (needing further branching) but its bound was still 7, would it still have been correct to prune it once the incumbent reached 8? Why or why not?

5 • Follow-through **Transfer — Where Else?**

The bounding argument you just used by hand is exactly what lets Gurobi report a solution as *provably* optimal – a 0% optimality gap – rather than merely "the best one found so far," the same guarantee Week 4's local search methods (min-conflicts, large neighbourhood search) explicitly do *not* give you. It also plants the seed for Week 10: the LP relaxation solved at the optimal node is precisely where sensitivity analysis and shadow prices come from, though this week stops at "it gives a bound," not "what else does it tell you."

7.3 First Models in gurobipy: Assignment, Knapsack, and Production Planning

1 • Pre-instruction **Why It Matters**

So far in this course, one CSP has meant one CP-SAT script. This unit hands you a second toolkit for two problems you already know by name – knapsack and assignment walked through MiniZinc in Week 6 – plus one brand-new one, production planning, all three expressed with a linear objective and linear constraints and solved by a genuinely different kind of engine.

1 • Pre-instruction **Prior Knowledge**

Week 1 Unit 4's CP-SAT building blocks – `CpModel()`, `NewIntVar`, `Add`, `Solve` – are the direct scaffold for `gurobipy`'s equivalents introduced below. Unit 1's knapsack IP formulation becomes, almost line for line, this unit's `knapsack.py`.

gurobipy: Gurobi's Python interface, used in this course through the `gurobipy` package to build and solve LP/MIP models.

Unit 1 and Unit 2 stayed entirely on paper, deliberately – the mathematics of an LP relaxation and a branch-and-bound proof needed to be visible without a solver's machinery hiding any of it. This unit finally hands that mathematics to a real solver: **gurobipy**, Gurobi's Python interface. Every model you build with it follows the same broad recipe Week 1 Unit 4 already taught you for CP-SAT: create a model object, declare decision variables, add constraints, then ask a solver to search for an assignment – only now the objective is not optional, and the variables can be continuous.

7.3.1 The building blocks, side by side with CP-SAT

Four pieces of `gurobipy`'s API cover almost everything this unit does with it, each one a close cousin of a CP-SAT piece you already know. A `gp.Model()` instance (compare `cp_model.CpModel()`) is an empty container you populate with variables and constraints. Decision variables come from `model.addVar` or, for a whole family of variables at once, `model.addVars` (compare `model.NewIntVar`). Linear constraints come from `model.addConstr` (compare `model.Add`). The one genuinely new piece is `model.setObjective`, called with an expression and either `GRB.MAXIMIZE` or `GRB.MINIMIZE` – Week 1's CP-SAT models never called anything like it, because a pure CSP carries no objective at all; an LP or MIP always does. Solving is `model.optimize()` (compare `solver.Solve`), and reading results back happens through `model.ObjVal` (the objective's optimal value) and a variable's `.X` attribute (compare `solver.Value`).

Three values fill `vtype`, the argument selecting each decision variable's kind: `GRB.CONTINUOUS` (the LP-relaxation-style variable Unit 1 introduced), `GRB.INTEGER`, and `GRB.BINARY` – the `{0,1}` case both knapsack and assignment need below.

Definition: vtype

The `gurobipy` variable-type argument selecting `GRB.CONTINUOUS`, `GRB.INTEGER`, or `GRB.BINARY` for a decision variable.

Reading a solve's outcome uses the same one-line habit Week 1 Unit 4 already built: check `model.Status == GRB.OPTIMAL` (compare `status == cp_model.OPTIMAL`) before trusting any variable's `.X` value.

Common Pitfall

A frequent mistake when moving from a paper formulation to `gurobipy` code is writing a comparison such as `x[item] <= 1` on its own line and expecting it to enforce anything. `gurobipy` overloads comparison operators to *build* a constraint object, not to *evaluate* one – written outside `model.addConstr(...)`, the comparison produces an inert, unattached constraint object and does nothing to the model, the same trap Week 1 Unit 4 warned you about for CP-SAT's `model.Add(...)`. The comparison only becomes part of the model once it is passed to `addConstr(...)` (or `addConstrs` for a whole family at

once).

7.3.2 Knapsack: an IP and its relaxation, both real code

Unit 1's exact A/B/C knapsack instance becomes two small `gurobipy` models – the same three items and the same capacity constraint, reused between them through one shared function. The `GRB.BINARY` variables and the model that will hold them:

```
model = gp.Model("knapsack_ip")
x = {item: model.addVar(vtype=GRB.BINARY, name=item) for item in ITEMS}
```

The single capacity constraint – reused unchanged by both the IP solve and the LP relaxation below, exactly the way Unit 1's paper formulation kept the constraint identical between the two:

```
model.addConstr(
    gp.quicksum(WEIGHT[item] * x[item] for item in ITEMS) <= CAPACITY,
    name="capacity",
)
```

Solving the IP and reading back the result:

```
model.optimize()
status = "OPTIMAL" if model.Status == GRB.OPTIMAL else str(model.Status)
print(f"Status: {status}")
print(f"Optimal value: {model.ObjVal}")
for item in ITEMS:
    print(f" {item}: {x[item].X}")
```

Expected Output

```
Status: OPTIMAL
Optimal value: 8.0
  A: 1.0
  B: 0.0
  C: 1.0
```

This is the real, captured output of the code above – an optimal value of 8.0, with A and C packed and B left out, exactly Unit 1's by-hand enumeration result $\{A, C\}$. The LP relaxation rebuilds the same three variables as `GRB.CONTINUOUS`, bounded 0 to 1, and solves again:

```
model = gp.Model("knapsack_lp_relaxation")
x = {
    item: model.addVar(vtype=GRB.CONTINUOUS, lb=0, ub=1, name=item)
    for item in ITEMS
}
add_capacity_constraint(model, x)
model.setObjective(
    gp.quicksum(VALUE[item] * x[item] for item in ITEMS), GRB.MAXIMIZE
)
model.optimize()
print(f"Relaxed optimal value: {model.ObjVal}")
for item in ITEMS:
    print(f" {item}: {x[item].X}")
```

Expected Output

```
Relaxed optimal value: 8.25
  A: 1.0
  B: 1.0
  C: 0.25
```

The relaxed value, 8.25, and the fractional `C: 0.25` match Unit 1's hand-computed relaxation exactly – a real solver run confirming a calculation you already did on paper.

7.3.3 Assignment: two constraint families, one linear cost

Unit 3's worked cost matrix pairs four workers (W1–W4) with four tasks (Ta–Td): a cheap diagonal (4, 3, 2, 5) and every off-diagonal cost fixed at 9. Binary variables, one per worker-task pair, indexed directly by `addVars`:

```
model = gp.Model("assignment")
x = model.addVars(WORKERS, TASKS, vtype=GRB.BINARY, name="x")
```

Two constraint families state the pairing rule – every worker does exactly one task, and every task gets exactly one worker:

```
for worker in WORKERS:
   model.addConstr(gp.quicksum(x[worker, task] for task in TASKS) == 1)
for task in TASKS:
   model.addConstr(gp.quicksum(x[worker, task] for worker in WORKERS) == 1)
```

Solving and reporting the chosen pairs alongside the total cost:

```
model.optimize()
assignment = {}
for worker in WORKERS:
   for task in TASKS:
      if x[worker, task].X > 0.5:
         assignment[worker] = task
         print(f"{worker} -> {task}")
print(f"Total cost: {model.ObjVal}")
```

Expected Output

```
W1 -> Ta
W2 -> Tb
W3 -> Tc
W4 -> Td
Total cost: 14.0
```

The unique optimal assignment is the matrix's diagonal, total cost 14 – exactly as expected, since substituting any off-diagonal pairing would force at least one cost-9 entry into the sum in place of a much smaller diagonal value.

Analogy

Read the assignment problem as assigning teaching assistants to exam-invigilation slots at minimum total travel cost: each assistant must cover exactly one slot, each slot must be covered by exactly one assistant, and the pairing with the lowest total cost wins – exactly the two constraint families and the linear objective above, with workers standing in for assistants and tasks for slots.

7.3.4 Production planning: a brand-new problem this week

A small factory makes three products, each with a per-unit profit and a per-unit consumption of two shared, budget-limited resources – labour hours and machine hours – and production quantities must be integers (no shipping half a manufactured unit). Integer decision variables, one per product:

```
model = gp.Model("production_planning")
x = {
   product: model.addVar(vtype=GRB.INTEGER, lb=0, name=product)
   for product in PRODUCTS
}
```

Both resource-budget constraints:

```
model.addConstr(
   gp.quicksum(LABOUR_USE[p] * x[p] for p in PRODUCTS) <= LABOUR_BUDGET,
   name="labour",
)
model.addConstr(
   gp.quicksum(MACHINE_USE[p] * x[p] for p in PRODUCTS) <= MACHINE_BUDGET,
   name="machine",
)
```

Solving and reporting the plan and the total profit:

```
model.optimize()
plan = {product: x[product].X for product in PRODUCTS}
for product in PRODUCTS:
   print(f"{product}: {plan[product]}")
print(f"Total profit: {model.ObjVal}")
```

restricted (size-limited) licence: Gurobi's default licence when no licence file is present: solves models up to 2000 variables and 2000 constraints, printed as a banner at the start of solver output.

Calling `optimize()` with no `gurobi.lic` file present starts every solve, this one included, by printing a **restricted (size-limited) licence** banner – the first lines of this model's real, captured solver log:

Expected Output

```
Restricted license - for non-production use only - expires 2026-11-23
Gurobi Optimizer version 12.0.3 build v12.0.3rc0 (mac64[arm] - Darwin
↪  25.6.0 25G83)
```

Caution

This banner is expected, unmodified output, not an error to chase down or hide – it is exactly what `gurobipy` prints whenever no licence file grants it a full commercial licence, and it says nothing about whether your model will solve correctly. The restricted licence only refuses models with more than 2000 variables or 2000 constraints; every model in this unit sits at a handful of variables and constraints each, far under that limit, so the solve proceeds exactly as normal once the banner has printed.

The plan and profit this same run reports, from the same captured file:

Expected Output

```
P1: 9.0
P2: 0.0
P3: 13.0
Total profit: 620.0
```

Nine units of product 1 and thirteen of product 3, none of product 2, for a total profit of 620 – read directly from the real solver run, since (unlike the knapsack and assignment instances above) no exact optimum for this instance was computed by hand in advance.

Tip

Gurobi's solved model can also report quantities named `Pi` and `RC` on a constraint or variable – dual values and reduced costs. This week only names them; Week 10 is where their meaning and use become the entire subject.

3 • Participation **Now You Try**

A partially written `gurobipy` script for the four-worker/ four-task assignment problem above is missing its "every task gets exactly one worker" constraints – only the "every worker does exactly one task" constraints are present. Write the missing constraint pattern, add it, run the completed script, and report the optimal total cost.

Feedback. The missing pattern is one constraint per task: `for j in tasks: model.addConstr(gp.quicksum(x[i, j] for i in workers) == 1)`. With both constraint families present, the script reports the unique optimal assignment – the matrix's diagonal – at total cost **14**, verifiable by hand from the cost matrix above, since every non-diagonal choice necessarily includes at least one cost-9 entry in place of a cost-$\leq$5 diagonal entry.

Check Yourself 7.3

Describe, in words, a `gurobipy` model (variables, objective, constraints, variable types) for a production-planning problem with two products: product 1 earns 40 profit per unit, needs 2 labour hours and 3 material units per unit; product 2 earns 30 profit per unit, needs 4 labour hours and 2 material units per unit; the factory has 40 labour hours and 60 material units available. State which variable type you would choose and why.

5 • Follow-through **Transfer — Where Else?**

The three modelling patterns you just used – a linear objective built with `setObjective`, constraint families built with `addVars`/`addConstr`/`quicksum`, and the choice between `BINARY`, `INTEGER` and `CONTINUOUS` `vtypes` – are exactly what Week 10 reuses without re-teaching (the same production-planning problem there gets a full sensitivity report), what Week 12 reuses for rostering's assignment-flavoured constraints, and what Week 14 reuses for facility location.

7.4 CP vs MIP: The Same Problem, Two Ways

1 • Pre-instruction — Why It Matters

You now own two solvers that both promise "the best answer, and a proof it is the best." A colleague hands you a new problem next month and asks which one to reach for – this unit gives you a way to answer that is not "whichever syntax comes to mind first."

1 • Pre-instruction — Prior Knowledge

Unit 3's `gurobipy` assignment model and Week 1 Unit 4's CP-SAT building blocks – this unit puts the two side by side on the exact same problem for the first time.

Unit 3 built the four-worker/four-task assignment problem entirely in `gurobipy`. This unit does not write new code – it takes that same problem and sketches, in words and API names only, how CP-SAT would model it instead, then compares the two side by side so the choice between them stops being a coin flip.

7.4.1 The same assignment problem, sketched in CP-SAT

Where Unit 3 used a binary variable `x[i, j]` for every worker-task pair, a CP-SAT model of the same problem more naturally uses one integer variable per worker, `assigned[i]`, with domain `{0..n-1}` standing for the task index that worker is given – a viewpoint choice in exactly Week 1 Unit 2's sense, and a different one from `gurobipy`'s pair-indexed binaries. A single `model.AddAllDifferent(assigned)` call directly captures "every task is used by at most one worker"; since the number of workers equals the number of tasks here, that also means "exactly one." Expressing the *cost* objective, however, needs an `AddElement`-style lookup construct, because the cost of worker i's choice depends on both i and the *value* that `assigned[i]` takes – a two-index lookup into a cost table, not a plain sum over pre-declared binaries.

Unit 3's `gurobipy` sketch, by contrast, needs binary `x[i, j]` and two families of linear equality constraints to state the same "each worker/task used exactly once" idea – no single global constraint call does it in one line – but the cost objective is a plain linear sum, `sum(cost[i][j] * x[i, j] for i, j in ...)`, with no lookup construct needed at all. Figure 7.3 places both sketches side by side.

Analogy

Choosing between CP-SAT and `gurobipy` is like choosing between two travel-planning tools: one tool is built to enforce complicated combination rules directly ("no two connecting flights may overlap," much like a global constraint); the other is built to optimize a cost number and tell you exactly how much cheaper a different budget would make your trip (foreshadowing Week 10) – you pick the tool that matches what you most need answered.

7.4.2 A working rule of thumb

Neither engine wins on every problem, and the assignment sketch above already shows why: each one expresses a different half of the same problem more directly.

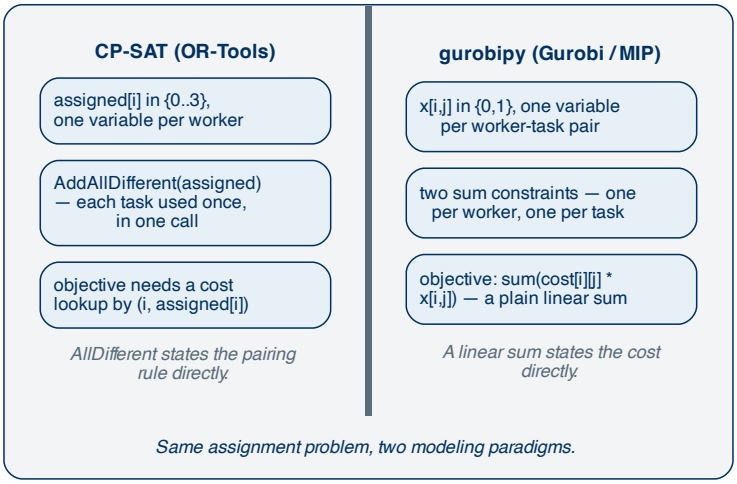


Figure 7.3: The same assignment problem expressed in CP-SAT and gurobipy.

CP-SAT is strong where a problem's structure is naturally combinatorial and expressible with a small number of global constraints – `AllDifferent` here, and later `Cumulative`, `NoOverlap`, `Circuit` – but it does not, out of the box, give you the sensitivity or what-if machinery Week 10 builds. `gurobipy`/MIP is strong where the objective and constraints are naturally linear and where a sensitivity report or a provable optimality gap on a cost or profit number matters; expressing some combinatorial relations, "all different" among them, needs extra binary variables and linearization rather than one direct call.

A common misconception at this point is assuming one engine is simply "better" than the other, in general. Neither is: the comparison above is not a scoreboard, it is a description of which half of a given problem each engine's native constructs reach for first. The right question is never "which solver is stronger" but "which half of *this* problem – its combinatorial structure or its cost objective – is harder to express, and which engine expresses that half directly."

Case Study 7.3: Reading the assignment sketch for the rule of thumb

Ask which model states "every task gets exactly one worker" more directly. CP-SAT's single `AddAllDifferent(assigned)` call states it in one line; `gurobipy`'s two families of sum constraints need one constraint per worker and one per task to say the same thing – CP-SAT wins this half. Now ask which model states the cost objective more directly. `gurobipy`'s `sum(cost[i][j] * x[i, j] ...)` is a plain linear sum; CP-SAT's cost would need an element-style lookup construct built on top of `assigned[i]` – `gurobipy` wins this half. Both answers follow directly from reading each sketch's constructs, not from any general ranking of the two engines.

Complexity (big-O)

Both engines share one property Week 4's local-search methods – min-conflicts, large neighbourhood search – explicitly do not give you: when either one reports "optimal," that is a proof, backed by Unit 2's bounding ar-

gument on the `gurobipy` side and by CP-SAT's own propagation-and-search proof machinery on the other, not a best-effort claim about the best solution tried so far.

3 • Participation **Now You Try**

Using the assignment problem from Unit 3, sketch (in words, no code) a CP-SAT model: which variables, which global constraint, and what would stand in for the objective. Then say which of the two models – this CP-SAT sketch or Unit 3's `gurobipy` model – more directly expresses "every task gets exactly one worker" as a single piece of the model, and which more directly expresses the cost to minimize.

Feedback. CP-SAT sketch: one variable `assigned[i]` per worker, domain `{0..3}` (the task index); `model.AddAllDifferent( assigned)` directly captures "every task is used by exactly one worker." The cost objective would need an element- or table-lookup style constraint, or auxiliary cost variables, since CP-SAT's objective is not naturally a lookup into a matrix by two indices. `AllDifferent` more directly expresses the "each task used once" idea in a single call than `gurobipy`'s two families of sum constraints do; conversely, `gurobipy`'s `sum(cost[i][j] * x[i, j] ...)` more directly expresses the cost objective than CP-SAT's lookup-based construction would – each paradigm wins on a different half of the same problem.

Check Yourself 7.4

State, in one or two sentences, a rule of thumb for choosing CP-SAT versus `gurobipy`/MIP for a new problem, and name one later week where the course combines the two rather than choosing between them.

5 • Follow-through **Transfer — Where Else?**

A one-sentence forward pointer, not developed further this week: some problems are best served by *combining* both engines rather than choosing one – Week 12's logic-based Benders decomposition is exactly that combination. The comparison you just made by hand is not academic – it is exactly the judgment call Week 12's CP–MIP hybrids ask you to make formally, and Week 15's benchmarking chapter asks you to back a solver choice with measured data instead of intuition alone.

Chapter Summary

Summary

This chapter gave you mathematical programming as a second full paradigm alongside constraint programming. Unit 1 defined a linear program as continuous variables, a linear objective, and linear constraints; an integer or mixed-integer program as an LP with some or all variables restricted to integer or

binary values; and LP relaxation as the LP obtained by dropping every integrality restriction, whose optimal value is always a valid bound on the original MIP's optimal value – illustrated on a three-item knapsack instance whose relaxed optimum (8.25) sat strictly above its true integer optimum (8). Unit 2 built branch-and-bound directly on that bound: branch on a fractional variable, solve each child's relaxation, and prune any child whose bound cannot beat the current incumbent – traced by hand on the same knapsack instance to a proven optimum of 8 after exploring only two children. Unit 3 turned both units' paper mathematics into real `gurobipy` code: you declared `CONTINUOUS`, `INTEGER` and `BINARY` variables, added linear constraints and a `setObjective` call, solved with `optimize()`, and read back `ObjVal` and each variable's `.X` – across the knapsack instance (confirming 8 and 8.25 from a real solver run), the four-worker/four-task assignment problem (optimal cost 14, the cost matrix's diagonal), and a new production-planning problem (profit 620), meeting the restricted size-limited licence banner along the way. Unit 4 placed a CP-SAT sketch of the assignment problem beside Unit 3's `gurobipy` model: `AllDifferent` states the pairing rule directly where `gurobipy` needs two constraint families, while `gurobipy`'s linear sum states the cost objective directly where CP-SAT would need a lookup construct – each paradigm winning on a different half of the same problem, and a working rule of thumb for choosing between them. Next week returns to CP-SAT and finally gives it a formal objective for over-constrained and weighted problems; Week 10 returns to this week's LP relaxation and its exact three problems and turns `Pi` and `RC`, only named in passing this week, into that entire chapter's subject.

Check Yourself Answers

Question (Unit 1 – a second knapsack instance, capacity 7). IP: maximize $5x_D + 6x_E + 8x_F$ subject to $3x_D + 4x_E + 5x_F \leq 7$, $x_D, x_E, x_F \in \{0, 1\}$; the LP relaxation replaces the domain restriction with $0 \leq x_D, x_E, x_F \leq 1$, keeping the objective and constraint identical. Ranking by value-per-weight ratio: D $= 5/3 \approx 1.667$, F $= 8/5 = 1.6$, E $= 6/4 = 1.5$ – order D, F, E. Greedy fill: $x_D = 1$ (weight 3, 4 left), then F's full weight of 5 exceeds the remaining 4, so $x_F = 4/5 = 0.8$ (weight 4, value 6.4) – relaxed optimum $5 + 6.4 = 11.4$; the fractional variable is x_F.

Question (Unit 2 – would pruning still be correct if the $x_C = 0$ child's bound of 7 had not been integral?). Yes – pruning only depends on the bound being no better than the incumbent, never on whether the bound is itself already integral. A fractional bound of 7 is still a valid upper bound on everything reachable inside that subtree (Unit 1's relaxation argument holds at every node, not only integer-feasible ones), and 7 cannot beat an incumbent of 8 no matter how the branch's remaining fractional variables would eventually have been resolved.

Question (Unit 3 – a two-product production-planning model, described in words). Variables x_1, x_2 = units of product 1 and product 2 produced; objective maximize $40x_1 + 30x_2$; constraints $2x_1 + 4x_2 \leq 40$ (labour), $3x_1 + 2x_2 \leq 60$ (material), $x_1, x_2 \geq 0$. `GRB.INTEGER` is the natural variable-type choice, since fractional units generally cannot be shipped for a discrete manufactured good;

`GRB.CONTINUOUS` would only be defensible if fractional batches were themselves meaningful – for example, a bulk, divisible product rather than a discrete unit.

Question (Unit 4 – a rule of thumb for choosing CP-SAT versus gurobipy/MIP). Choose CP-SAT when a problem's structure is naturally combinatorial and expressible with a small number of global constraints and no sensitivity or what-if machinery is needed; choose `gurobipy`/MIP when the objective and constraints are naturally linear and a provable optimality gap, or later a sensitivity report (Week 10), is wanted. Week 12's CP–MIP hybrids – logic-based Benders decomposition – combine both engines rather than picking one.